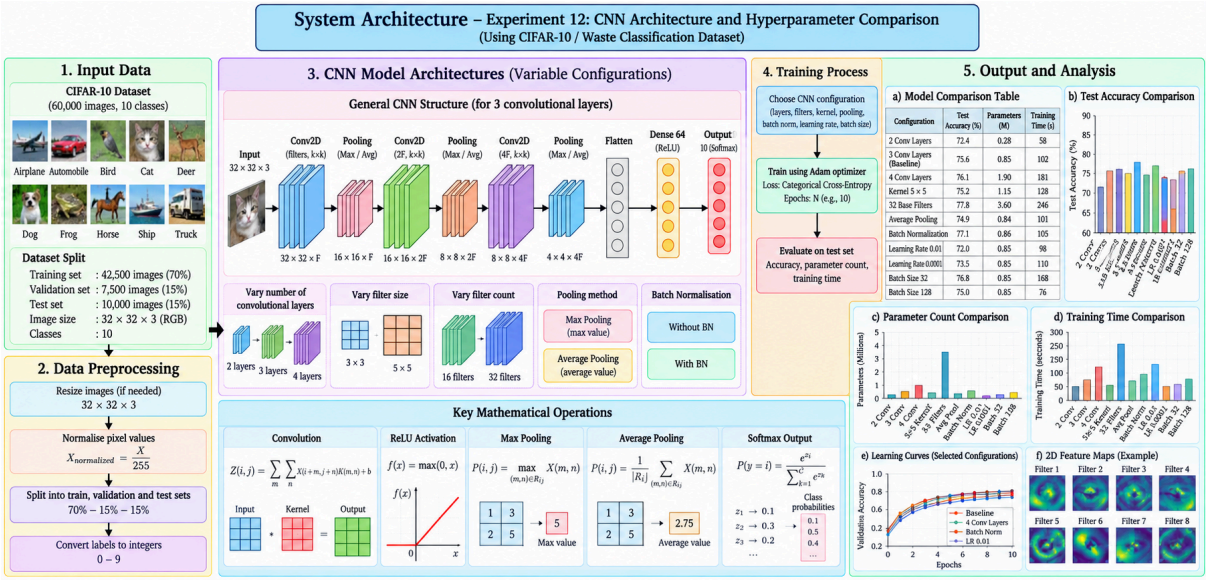
# Experiment 12: CNN Architecture and Hyperparameter Comparison

## 1. Aim

To compare different Convolutional Neural Network (CNN) architectures and hyperparameters for image classification using TensorFlow and Keras. To investigate the effects of convolutional layer depth, filter size, filter count, pooling methods, batch normalization, learning rate, and batch size on classification accuracy, model parameter count, and training time, and identify a suitable CNN configuration for a low-cost edge device.



## 2. Mathematical Formulas

### 1. Convolution Operation

$$\boxed{Z(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b}$$

### 2. ReLU Activation Function

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

### 3. Max Pooling

$$\boxed{P(i,j) = \max_{(m,n) \in R_{ij}} X(m,n)}$$

### 4. Average Pooling

$$\boxed{P(i,j) = \frac{1}{|R_{ij}|} \sum_{(m,n) \in R_{ij}} X(m,n)}$$

### 5. Batch Normalization

$$\boxed{\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}}$$

$$\boxed{y = \gamma \hat{x} + \beta}$$

### 6. Softmax Function

$$\boxed{P(y=i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}}$$

### 7. Categorical Classification Loss

For integer class labels, sparse categorical cross-entropy is:

$$\boxed{L = -\frac{1}{N} \sum_{i=1}^{N} \log(p_{i,y_i})}$$

```python
# Experiment 12: CNN Architecture and Hyperparameter Comparison

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import time

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

np.random.seed(42)
tf.random.set_seed(42)

# ======================================================
# 1. LOAD AND PREPROCESS CIFAR-10
# ======================================================

(X_all, y_all), (X_test_all, y_test_all) = (
    tf.keras.datasets.cifar10.load_data()
)

y_all = y_all.ravel()
y_test_all = y_test_all.ravel()

# Use a subset to reduce Colab execution time
X_all = X_all[:12000].astype("float32") / 255.0
y_all = y_all[:12000]

X_test = X_test_all[:2000].astype("float32") / 255.0
y_test = y_test_all[:2000]

X_train, X_val, y_train, y_val = train_test_split(
    X_all, y_all,
    test_size=0.20,
    random_state=42,
    stratify=y_all
)

print("Training images:", len(X_train))
print("Validation images:", len(X_val))
print("Test images:", len(X_test))
print("Image shape:", X_train.shape[1:])

# ======================================================
# 2. DEFINE CNN ARCHITECTURE
# ======================================================

def build_cnn(config):
    inputs = tf.keras.layers.Input(shape=(32, 32, 3))
    x = inputs

    filters = config["filters"]
    kernel_size = config["kernel"]
    num_layers = config["layers"]

    for i in range(num_layers):
        x = tf.keras.layers.Conv2D(
            filters=filters * (2 ** i),
            kernel_size=kernel_size,
            padding="same",
            use_bias=not config["batch_norm"]
        )(x)
```

```
        if config["batch_norm"]:
            x = tf.keras.layers.BatchNormalization()(x)

        x = tf.keras.layers.Activation("relu")(x)

        if config["pooling"] == "max":
            x = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(x)
        else:
            x = tf.keras.layers.AveragePooling2D(pool_size=(2, 2))(x)

    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dense(64, activation="relu")(x)
    outputs = tf.keras.layers.Dense(10, activation="softmax")(x)

    model = tf.keras.Model(inputs, outputs)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=config["lr"]
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

# ======================================================
# 3. DEFINE EXPERIMENT CONFIGURATIONS
# ======================================================

configs = [
    {
        "name": "Baseline",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "2 Conv Layers",
        "layers": 2, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "4 Conv Layers",
        "layers": 4, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "Kernel 5x5",
        "layers": 3, "kernel": 5, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "32 Base Filters",
        "layers": 3, "kernel": 3, "filters": 32,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "Average Pooling",
        "layers": 3, "kernel": 3, "filters": 16,
```

```
        "pooling": "avg", "batch_norm": False,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "Batch Normalization",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": True,
        "lr": 0.001, "batch_size": 64
    },
    {
        "name": "Learning Rate 0.01",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.01, "batch_size": 64
    },
    {
        "name": "Learning Rate 0.0001",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.0001, "batch_size": 64
    },
    {
        "name": "Batch Size 32",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 32
    },
    {
        "name": "Batch Size 128",
        "layers": 3, "kernel": 3, "filters": 16,
        "pooling": "max", "batch_norm": False,
        "lr": 0.001, "batch_size": 128
    }
]

# ======================================================
# 4. TRAIN AND EVALUATE ALL CONFIGURATIONS
# ======================================================

results = []
histories = {}

EPOCHS = 5  # Increase to 10 or more for deeper comparison

for config in configs:
    print("\nTraining:", config["name"])

    tf.keras.backend.clear_session()
    tf.random.set_seed(42)

    model = build_cnn(config)

    start_time = time.perf_counter()

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        batch_size=config["batch_size"],
        verbose=0
    )

    training_time = time.perf_counter() - start_time
```

```
    test_loss, test_accuracy = model.evaluate(
        X_test, y_test, verbose=0
    )

    results.append({
        "Configuration": config["name"],
        "Conv Layers": config["layers"],
        "Kernel": f'{config["kernel"]}x{config["kernel"]}',
        "Base Filters": config["filters"],
        "Pooling": config["pooling"],
        "Batch Norm": config["batch_norm"],
        "Learning Rate": config["lr"],
        "Batch Size": config["batch_size"],
        "Parameters": model.count_params(),
        "Test Accuracy (%)": test_accuracy * 100,
        "Training Time (s)": training_time,
        "Final Validation Accuracy (%)":
            history.history["val_accuracy"][-1] * 100
    })

    histories[config["name"]] = history.history

    print("Parameters:", model.count_params())
    print("Test accuracy:", round(test_accuracy * 100, 2), "%")
    print("Training time:", round(training_time, 2), "seconds")

# ======================================================
# 5. DISPLAY COMPARISON TABLE
# ======================================================

results_df = pd.DataFrame(results)

print("\nFULL HYPERPARAMETER COMPARISON")
print(results_df.round(3).to_string(index=False))

# ======================================================
# 6. PLOT ACCURACY COMPARISON
# ======================================================

plt.figure(figsize=(12, 6))

plt.bar(
    results_df["Configuration"],
    results_df["Test Accuracy (%)"]
)

plt.ylabel("Test Accuracy (%)")
plt.title("CNN Test Accuracy by Configuration")
plt.xticks(rotation=55, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# ======================================================
# 7. PLOT PARAMETER COUNT
# ======================================================

plt.figure(figsize=(12, 6))

plt.bar(
    results_df["Configuration"],
    results_df["Parameters"]
)
```

```
plt.ylabel("Number of Parameters")
plt.title("CNN Model Size Comparison")
plt.xticks(rotation=55, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# ====================================================
# 8. PLOT TRAINING TIME
# ====================================================

plt.figure(figsize=(12, 6))

plt.bar(
    results_df["Configuration"],
    results_df["Training Time (s)"]
)

plt.ylabel("Training Time (seconds)")
plt.title("CNN Training Time Comparison")
plt.xticks(rotation=55, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# ====================================================
# 9. COMPARE LEARNING CURVES
# ====================================================

plt.figure(figsize=(10, 6))

for name in [
    "Baseline",
    "Learning Rate 0.01",
    "Learning Rate 0.0001",
    "Batch Normalization"
]:
    plt.plot(
        histories[name]["val_accuracy"],
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Across Selected Configurations")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ====================================================
# 10. FIND A SMALL, ACCURATE MODEL
# ====================================================

# Illustrative trade-off: models within 2 percentage points
# of the best test accuracy, then choose the smallest.
best_accuracy = results_df["Test Accuracy (%)"].max()

eligible = results_df[
    results_df["Test Accuracy (%)"] >= best_accuracy - 2
]

recommended = eligible.sort_values(
    "Parameters"
```

```
).iloc[0]

print("\nBest test accuracy:")
print(results_df.loc[
    results_df["Test Accuracy (%)"].idxmax(),
    ["Configuration", "Test Accuracy (%)", "Parameters"]
].to_string())

print("\nSmallest model within 2 percentage points of best accuracy:")
print(recommended[
    ["Configuration", "Test Accuracy (%)",
     "Parameters", "Training Time (s)"]
].to_string())
```

## OUTPUT

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1749s 10us/step
Training images: 9600
Validation images: 2400
Test images: 2000
Image shape: (32, 32, 3)

Training: Baseline
Parameters: 28394
Test accuracy: 38.55 %
Training time: 85.11 seconds

Training: 2 Conv Layers
Parameters: 7850
Test accuracy: 30.8 %
Training time: 47.09 seconds

Training: 4 Conv Layers
Parameters: 106346
Test accuracy: 48.85 %
Training time: 74.27 seconds

Training: Kernel 5x5
Parameters: 70122
Test accuracy: 41.5 %
Training time: 129.26 seconds

Training: 32 Base Filters
Parameters: 102154
Test accuracy: 44.3 %
Training time: 171.29 seconds

Training: Average Pooling
Parameters: 28394
Test accuracy: 34.9 %
Training time: 50.41 seconds

Training: Batch Normalization
Parameters: 28730
Test accuracy: 48.0 %
Training time: 83.02 seconds

Training: Learning Rate 0.01
Parameters: 28394
Test accuracy: 36.5 %
Training time: 76.02 seconds

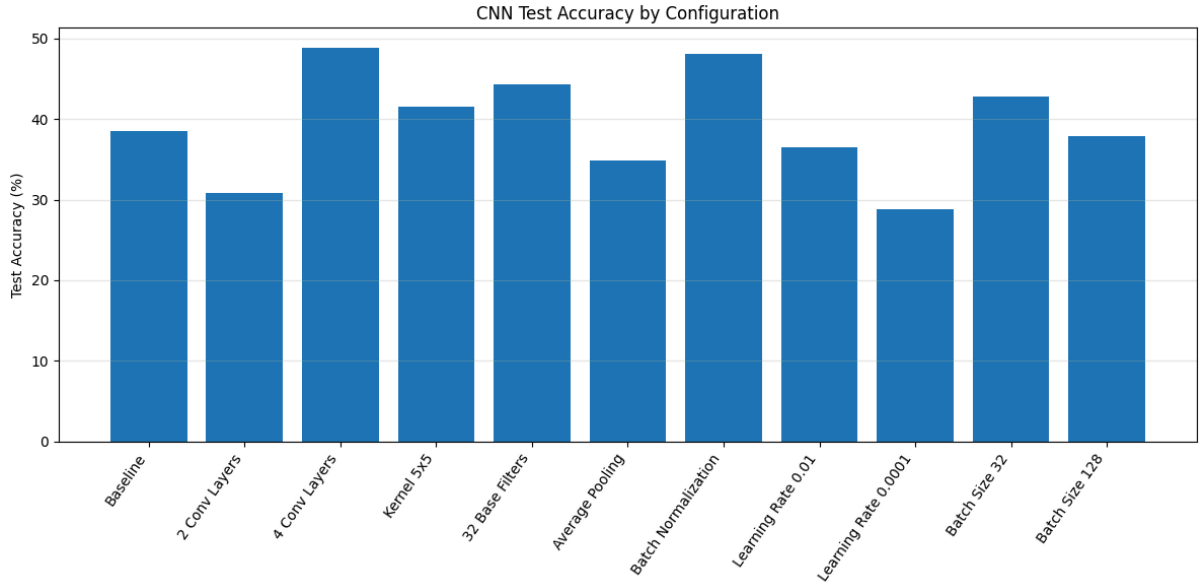
Training: Learning Rate 0.0001
Parameters: 28394
Test accuracy: 28.85 %
Training time: 57.38 seconds

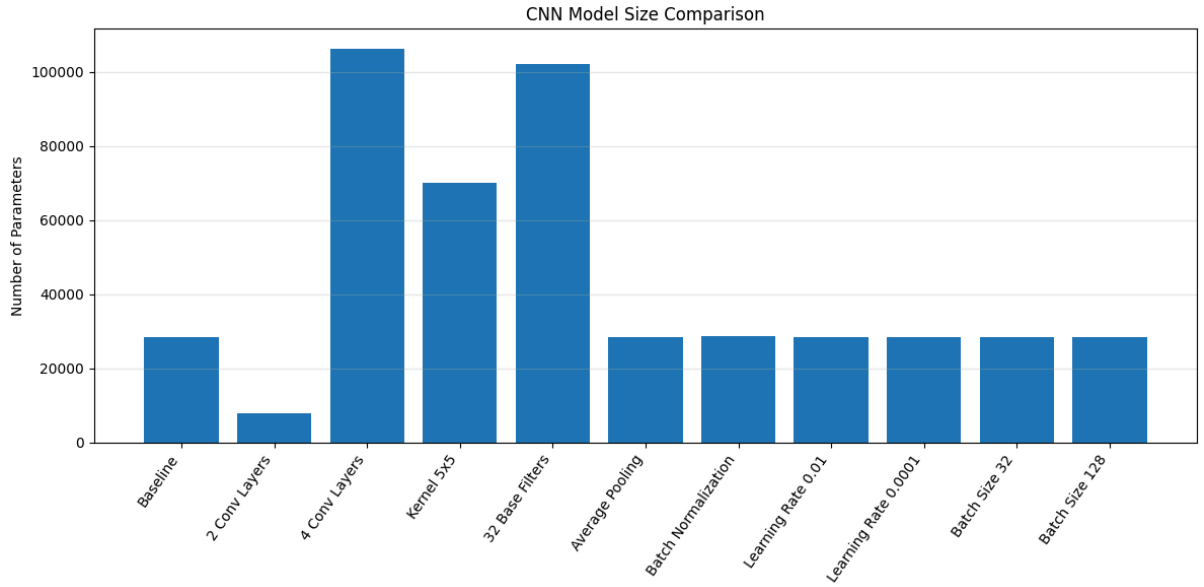
Training: Batch Size 32
Parameters: 28394
Test accuracy: 42.8 %
Training time: 87.14 seconds

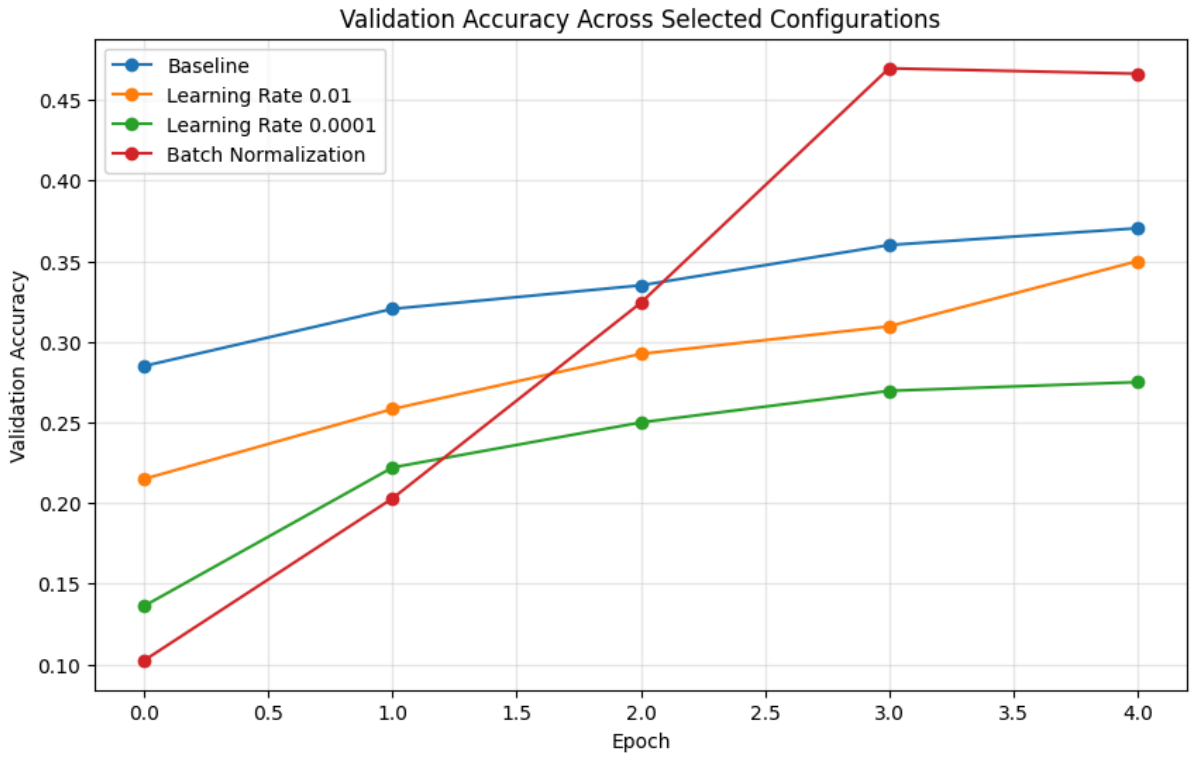
Training: Batch Size 128
Parameters: 28394
Test accuracy: 37.85 %
Training time: 83.64 seconds
```

FULL HYPERPARAMETER COMPARISON

| Configuration | Conv Layers | Kernel | Base Filters | Pooling | Batch Norm | Learning Rate | Batch Size | Parameters | Test Accuracy (%) | Training Time (s) | Final Validation Accuracy (%) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Baseline | 3 | 3x3 | 16 | max | False | 0.001 | 64 | 28394 | 38.55 | 85.113 | 37.042 |
| 2 Conv Layers | 2 | 3x3 | 16 | max | False | 0.001 | 64 | 7850 | 30.80 | 47.087 | 30.708 |
| 4 Conv Layers | 4 | 3x3 | 16 | max | False | 0.001 | 64 | 106346 | 48.85 | 74.267 | 47.458 |
| Kernel 5x5 | 3 | 5x5 | 16 | max | False | 0.001 | 64 | 70122 | 41.50 | 129.257 | 40.208 |
| 32 Base Filters | 3 | 3x3 | 32 | max | False | 0.001 | 64 | 102154 | 44.30 | 171.290 | 43.958 |
| Average Pooling | 3 | 3x3 | 16 | avg | False | 0.001 | 64 | 28394 | 34.90 | 50.407 | 33.333 |
| Batch Normalization | 3 | 3x3 | 16 | max | True | 0.001 | 64 | 28730 | 48.00 | 83.021 | 46.625 |
| Learning Rate 0.01 | 3 | 3x3 | 16 | max | False | 0.010 | 64 | 28394 | 36.50 | 76.017 | 35.000 |
| Learning Rate 0.0001 | 3 | 3x3 | 16 | max | False | 0.000 | 64 | 28394 | 28.85 | 57.382 | 27.500 |
| Batch Size 32 | 3 | 3x3 | 16 | max | False | 0.001 | 32 | 28394 | 42.80 | 87.144 | 43.583 |
| Batch Size 128 | 3 | 3x3 | 16 | max | False | 0.001 | 128 | 28394 | 37.85 | 83.642 | 37.125 |






Best test accuracy:

| | |
|---|---|
| Configuration | 4 Conv Layers |
| Test Accuracy (%) | 48.85 |
| Parameters | 106346 |

Smallest model within 2 percentage points of best accuracy:

| | |
|---|---|
| Configuration | Batch Normalization |
| Test Accuracy (%) | 47.999999 |
| Parameters | 28730 |
| Training Time (s) | 83.020552 |

## 5. RESULT

Different CNN architectures and hyperparameters were implemented and compared using TensorFlow and Keras on the CIFAR-10 dataset. The experiment investigated convolutional depth, filter size, filter count, pooling method, batch normalization, learning rate, and batch size. Each configuration was evaluated using test accuracy, parameter count, and training time, with results visualized using Matplotlib. The comparison helps identify an appropriate balance between classification performance and model complexity for a low-cost edge device.